

ADVANCED DATA MANAGEMENT 2024/2025
February 6, 2025
CLOSED BOOK

FIRST NAME:
SID:

LAST NAME:

Q1. (up to 10 lines) [**only for COMPUTER ENGINEERING STUDENTS**]
Explain the main differences between frameworks for parallel computing and NoSQL systems.

Q2. (up to 10 lines)
Explain the notion of aggregate: (a) what it represents at the logical level, (b) what it represents at the physical level; (c) which fields of an aggregate are visible at the system side, for each NoSQL data model introduced in the course (key-value, column-family, document-based).

Q3. (up to 10 lines)

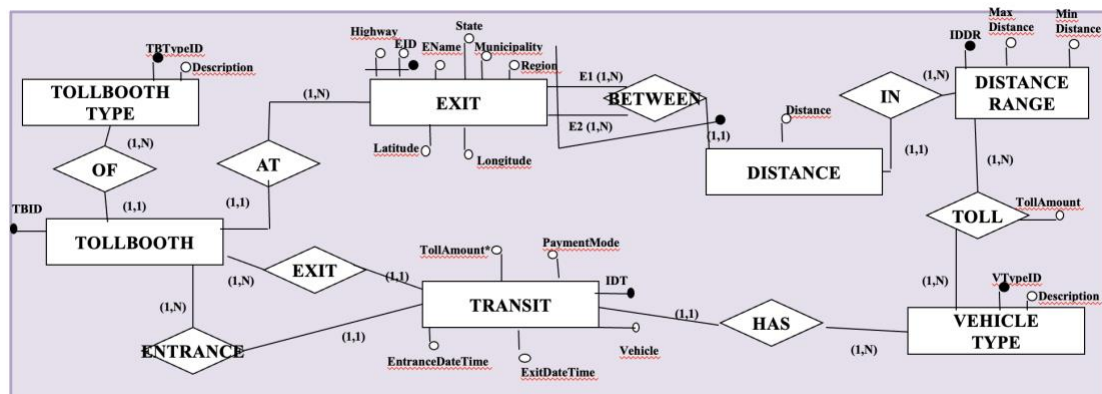
Discuss *hash-based partitioning over a ring*, taking Cassandra as a reference example.

Q4. (up to 10 lines) [**only for COMPUTER SCIENCE STUDENTS**]

Define the notion of resource for RDF, presenting an example.

Exercise 1 [NoSQL aggregate-oriented data stores]

Consider the following entity relationship diagram:



The workload is the following:

- **Query 1:** Determine the total toll amount for transits that occurred on a specific date (consider only transits that started and ended in the same day).
- **Query 2:** Determine all the vehicles that passed in entrance through a specific tollbooth in a given date.
- **Query 3:** Determine all the tollbooths of a certain type, located in a given municipality.

Based on the previous ER diagram and workload:

1. Design, using the aggregate-oriented logical design methodology, the aggregates needed for answering the given workload queries. Present the output of each step of the methodology.

Queries

Q1

Q2

Q3

Annotation: please annotate the input ER diagram

Aggregates

2. Suppose the selected system is Cassandra.
 - a. Revise each aggregate schema, in meta-notation, to obtain a set of aggregate schemas for Cassandra, specified in meta-notation, so that queries Q1, Q2, Q3 can be executed without the creation of any index and without the usage of the ALLOW FILTERING clause. Motivate your choices.
 - b. For each aggregate schema designed for Cassandra, choose a suitable partition key and a suitable primary key, motivating your choice.

- c. Present the CQL commands for the creation of one of the aggregates you have designed. If available, select one aggregate with at least one complex attribute.

3. Write in CQL query Q2.

Exercise 2 [NoSQL graph-based data stores]

Consider a graph containing information about airports and flights, according to the following schema information:

Node types and related properties

```
(:Airport {airport_id, abbr, name})  
(:City {name})  
(:State {code, name})  
(:Country {name, iso_code})  
(:Carrier {code, description})  
{:Flight {flight_number, year, month, day}}  
(:Aircraft {tail_number})  
(:Reason {code, description})
```

Relation types and related properties

```
(:City)-[:IN_COUNTRY]→(:Country)  
(:State)-[:IN_COUNTRY]→(:Country)  
(:City)-[:IN_STATE]→(:State)  
(:Airport)-[:IN_CITY]→(:City)  
(:Flight)-[:ORIGIN {taxi_time, dep_delay} | :DESTINATION {taxi_time,  
arr_delay}]→(:Airport)  
(:Flight)-[:CARRIER]→(:Carrier)  
(:Flight)-[:CANCELLED_BY|:DELAYED_BY {duration}]→(:Reason)  
(:Flight)-[:AIRCRAFT]→(:Aircraft)
```

1. Modify the schema to add information about Passengers and Flight Tickets (nodes, properties, relationships), using the previous format.
2. Formulate in Cypher the query to determine the passengers that have flown at least twice with AirFrance; for each of them, return the total number of flights and the average delay they have experienced.

Exercise 3 [Semantic data stores] [only for COMPUTER SCIENCE STUDENTS]

Consider the following extract from an RDF file:

```
ex:vacation1 ex:hasDestination ex:Paris .
ex:vacation1 ex:hasStartDate "2025-08-01" .
ex:vacation1 ex:hasEndDate "2025-08-10" .
ex:vacation1 ex:hasActivity ex:Sightseeing .
ex:vacation1 ex:hasAccommodation ex:HotelEiffel .
ex:vacation1 ex:hasTransport ex:FlightAF123 .

ex:vacation2 ex:hasDestination ex:Rome .
ex:vacation2 ex:hasStartDate "2025-07-20" .
ex:vacation2 ex:hasEndDate "2025-07-27" .
ex:vacation2 ex:hasActivity ex:MuseumTour .
ex:vacation2 ex:hasAccommodation ex:AirbnbColosseo .
ex:vacation2 ex:hasTransport ex:TrainFrecciarossa .

ex:Paris ex:hasCountry ex:France .
ex:Rome ex:hasCountry ex:Italy .

ex:Sightseeing ex:hasDescription "Visiting tourist attractions
and landmarks in the city." .
ex:MuseumTour ex:hasDescription "A guided tour of the city's most
famous museums." .

ex:HotelEiffel ex:hasType "Hotel" .
ex:AirbnbColosseo ex:hasType "Airbnb" .

ex:FlightAF123 ex:hasAirline ex:AirFrance .
ex:TrainFrecciarossa ex:hasOperator ex:Trenitalia .
```

1. Add the class specification for the class **ex:Vacation** and specify **ex:vacation2** as an instance of class **ex:Vacation**.
2. Define **ex:CityVacation** as a subclass of **ex:Chef** and define at least two properties with **ex:CityVacation** as domain, specifying also their corresponding range.

3. Is it possible to specify that a vacation corresponds to a single type of transportation? How?
4. Write a SPARQL query to retrieve all vacations that have a destination in Italy and include a sightseeing activity.
5. Write a SPARQL query to retrieve the name, the destination, and the transportation method (if available) for all the vacations that have a sightseeing activity or start in August.

Hint (Cheat sheet RDF/S and OWL)

- instance relation `rdf:type` (classes are instances of `rdfs:Class`)
- subclass relation `rdfs:subClassOf`
- domain of a property `rdfs:domain`
- range of a property `rdfs:range`
- subproperty `rdfs:subPropertyOf`
- functional property `owl:functionalProperty`
- inverse functional property `owl:inversefunctionalProperty`
- class disjointness `owl:disjointWith`
- instance equality and disequality `owl:sameAs` and `owl:differentFrom`